

Applied Learnings for Adversarial Distributed Systems

Methods, Explorations for everything
blockchains

BY
Supragya Raj

PUBLISHED IN THE WILD

Contents

1	ZKVMs: Zero Knowledge Virtual Machines	7
1.1	RISC-V as a common target	7
1.1.1	Why RISC-V though?	7
1.2	The general design of ZKVMs	8
1.3	Writing constraints for these instructions	8
1.4	Self-modifying code and dealing with it	8
1.5	Identifying the program	8
1.6	Proof generation without pre-compiles	8
1.7	Proof generation with pre-compiles	9
2	Version Control using Git	11
2.1	Using Git via SSH	11
3	Time, Synchrony and Node Failures	13
3.1	Key definitions	13
3.1.1	Consistency and Liveness	13
3.1.2	Node honesty	14
3.1.3	Network Synchrony	15
3.1.4	Consensus	15
3.1.5	Permissioning	15
3.1.6	Global Clock	15
3.1.7	Public Key Interchange Infrastructure	15
3.2	State Machine Replication	15
3.3	Dolev-Strong: Byzantine-broadcast, SMR, in a synchronous world	16
3.4	FLP: Can't handle even 1 faulty node in asynchronous world	16
3.4.1	The FLP Impossibility Claim	16
3.4.2	Key concepts	17
3.4.3	Consensus, System Configurations, Events and Schedules	18
3.4.4	Lemma 1: Disjoint Schedules Commute	18
3.4.5	Lemma 2: There exists a bi-valent initial configuration	18
3.4.6	Lemma 3: All roads lead to indecision	19
3.4.7	Proving the theorem	20
3.4.8	Conclusion	20
3.5	The CAP principle	21
4	Ethereum	23
4.1	Legacy Ethereum (Ethereum 1.0)	23
4.1.1	Recursive Length Prefix (RLP) encoding	23
4.1.2	How to deploy a contract: CREATE vs CREATE2	23
4.1.3	Wintermute Hack (2022): CREATE and EIP-155 for transaction signatures	24
4.1.4	Accessing Historical Block Hashes: EIP: 2935	25

4.2	Ethereum after Merge (Ethereum 2.0)	26
4.2.1	The Proof-of-Stake Block	26
4.2.2	Finality Metalayer: Casper FFG	26
4.3	Exercise Solutions	27
5	Solana	29
5.1	The transaction and the block	29
5.1.1	Transactions and "blockhash queue"	29
5.2	The Transaction Processing Unit (TPU)	30
5.3	Proof of History: VDF based transaction ordering	30
5.4	Entries, Slots, Bank	30
5.5	State proofs in Solana	31
5.5.1	Homomorphic state commitment: <code>AccountsLatticeHash</code>	31
5.5.2	Post-SIMD-223: <code>BankHash</code> composition	33
6	General Concepts in Adversarial Distributed Systems	35
6.1	Proof of Stake: The Nothing at Stake problem	35
6.1.1	No penalties = No canonicalization	35
6.1.2	Example: The 1% attacker	36
6.1.3	The Tragedy of the Commons	36
6.1.4	Simulating the "wrong choice is costly"	36
6.2	Wealth Inequality and the Gini Coefficient	36
6.3	Miner's Extractable Value (MEV)	37
6.3.1	Basic on-chain DeX based arbitrage	37
6.4	The ever growing state, state expiry	37
6.5	Rebuilding the chain / Chain Derivation	37
6.6	Supporting a different virtual machine	37
7	Layer 2 Scaling Solutions	39
7.1	General classification	39
7.2	Where sits the enormous data?	39
7.3	Plasma: Each one save one	40
7.3.1	Operator misbehaviors and exits	40
7.3.2	Plasma Cash	40
7.4	Bottlenecked by DA	40
7.5	Rollups and levels of confirmation	41
7.6	Faster Layer 2 Interoperability	41
8	An on-chain system for asynchronous transactions	43
8.1	Definitions	43
8.2	Time Turner	44
8.3	Asynchronous Execution Protocol (AEP)	44
8.3.1	Inter-Intent Dependencies	44
8.3.2	Intra-Intent Dependencies	44
8.3.3	Anatomy of an Action	45
8.3.4	Execution Forking for optimal fee	45
8.3.5	DApps' "Forking Loss" compensation	45
8.3.6	Intent grieving by solvers	46
8.3.7	Execution Payments	46
8.3.8	Intent Registration	46
8.3.9	Intent halt due to EOW collapse	46
8.3.10	On Options default by dApps or solvers	46
9	Golang	47

10 Rust	49
10.1 Iterators	49
11 Solidity	51
11.1 The memory model	51
11.1.1 Memory types: <code>calldata</code> vs <code>memory</code> vs <code>storage</code>	51
11.1.2 The on-chain storage	51
11.2 Function and their modifiers	52
11.2.1 Visibility Modifiers: <code>private</code> , <code>internal</code> , <code>external</code> , <code>public</code>	52
11.2.2 Function Modifiers for preconditions and postconditions	52
11.2.3 Special modifiers: <code>view</code> , <code>pure</code> and <code>payable</code>	53
11.3 The EIP-721 Standard: Non-Fungible Tokens (NFTs)	53
11.4 Exercise Solutions	54
12 Binary Search	55
12.1 Basic: Minimize k such that $f(k)$ is true	55
12.2 Advanced: Search spaces with <code>feasible(x) -> bool</code>	55
12.2.1 Problem [SeqShip]: Sequential objects shipping within D days	55
12.2.2 Problem [KokoBanana]: Monkey eating piles of bananas	56
12.2.3 Problem [SplitM]: Split array into m subarrays	56
12.3 Advanced: Search spaces with <code>enough(x) -> bool</code>	57
12.3.1 Problem [KSmallMul]: K-th smallest number in multiplication table	57
13 Dynamic Programming	59
13.1 Memoization	59
13.2 Most important subproblems	60
Alphabetical Index	61

Chapter 1

ZKVMs: Zero Knowledge Virtual Machines

This chapter intends to give a brief overview of the zero-knowledge virtual machines and their current state of usability.

1.1 RISC-V as a common target

As soon as we find ourselves exploring ZK-VMs, we quickly find ourselves with a bunch of projects such as RISC-Zero, Succinct's SP1, Mozak, Polygon Miden, Jolt etc, all of which for some reason are RISC-V compatible. What that means is that they expect input instructions to be composed of RISC-V instruction set.

1.1.1 Why RISC-V though?

Modern compiler infrastructure such as that of LLVM makes the code generation process modular. A lot of programming languages can be supported if we target RISC-V, for example C++ through Clang, Rust through RustC, Golang through TinyGo can all convert into LLVM bytecode. This can at the end be compiled down to a well known small instruction set target such as RISC-V. RISC-V essentially is the 5th reduced-instruction-set-architecture that came out of UC Berkeley; hence the name. The main reason however that RISC-V becomes a compilation target of interest is because of two simple reasons apart from language-support: **minimalism** and **modularity**.

Minimalism in RISC-V: RISC-V is relatively simple ISA. It has 32 registers that can hold 32 bits each (`x0-x31`) alongside a special register `pc` that holds the address of currently executing instruction's address. The base 32-bit version of RISC-V dealing in integers is `riscvi` and has well-known simple instructions like `ADD`, `JAL`, `BNE` etc. All of these are relatively simple to describe.

Modularity in RISC-V: RISC-V provides a base, but also provides a bunch of useful extensions that one can opt-in to. For example, `+m` provides functionality such as multiplication (and division), `+f` provides floating point numbers, `+a` gives access to atomics among others. This modularity allows us to choose the tightest subset of features to implement for a ZKVM.

RISC-V compatible ZK-VMs are mostly targeted towards `riscvi+m`.

1.2 The general design of ZKVMs

1.3 Writing constraints for these instructions

1.4 Self-modifying code and dealing with it

1.5 Identifying the program

1.6 Proof generation without pre-compiles

While the numbers described below present the situation as seen in December 2024, they still give good information on what we can expect in the signature verification regime. ED25519 is one of the signature schemes used quite commonly in blockchains like Solana, Tendermint-based chains like Cosmos as well as Polkadot.

Test Description	Execution Trace Generation Time	RISC-0 Segments Count	Total Cycles	User Cycles	Proving Time
ED25519 1 signature no-precompile 3-byte msg	91ms	4	3.67M	3.12M	130s
ED25519 1 signature no-precompile 3000-byte msg	91ms	4	4.19M	3.48M	147s
ED25519 2 signatures no-precompile 3-byte msg	171ms	7	6.81M	6.06M	242s
ED25519 2 signatures no-precompile 3000-byte msg	191ms	8	7.60M	6.78M	270s

Analysis of the above gives us the following key points:

- The proof generation procedure is undertaken in two stages: a simulation phase where the execution "trace" is generated and the proof generation phase where the execution trace is proven.
- Compared to the actual proving time in each which includes both the stages, the first stage (execution "trace" generation) takes miniscule amount of time. For example in the test running 1 ed25519 signature verification in zkvm, 91ms were taken in execution "trace" generation step while around 130,000ms were taken for proof generation.
- On average, on a very beefy machine without GPU, Risc-0 can clock in around **28,000 instruction / seconds** of proving.
- In large calculations, the verification is broken down into multiple smaller proving segments, limiting on average, proving time **30s / segment, 8GB RAM usage / segment**. Maybe we can use parallelization of these segments for faster proving, but we need to see who "proves things". Also, segmentation of proof provides larger overhead in proof accumulation and continuation checks not accounted for in these benchmarks.

Another important test we run is for a hash function: SHA256. Alongside ED25519, SHA256 becomes the primary underlying algorithm used extensively in chains like Solana

Test Description	Execution Trace Generation Time	RISC-0 Segments Count	Total Cycles	User Cycles	Proving Time
SHA256 1 byte	3.21ms	1	131K	8.9K	4.5s
SHA256 10 byte	3.25ms	1	131K	8.9K	4.5s
SHA256 100 byte	3.39ms	1	131K	15.4K	4.62s
SHA256 1,000 byte	4.9ms	1	262K	100K	9.26s
SHA256 10,000 byte	20.5ms	1	1.05M	0.96M	37s
SHA256 100,000 byte	175.43ms	10	10.48M	9.5M	368s

1.7 Proof generation with pre-compiles

A few other things we learn from the above experiments:

- We find that zkVM needs to run at least 2^{17} **cycles** of instructions regardless of what our code does. If instructions are less, the rest of the rows are padded, which amounts to extra zkvm cycles.
- The growth of total cycles required for execution grows linearly with amount of bytes in question.

Chapter 2

Version Control using Git

2.1 Using Git via SSH

We can use GIT via a specific SSH command override, useful for when we want to use a specific key for git clone or git push. This can be done via:

```
GIT_SSH_COMMAND='ssh -i ~/.ssh/your_private_key' git push
```


Chapter 3

Time, Synchrony and Node Failures

This chapter focuses on establishing the foundational building blocks for talking about distributed systems in general. This chapter first establishes key definitions and scope of problems at hand. Later, explorations are presented while arguing about different trade-offs.

3.1 Key definitions

We begin by setting the stage up for problem at hand. In distributed systems, almost always, we are dealing with a distributed set of nodes that wants to present themselves as a single entity to the client. While earlier, this distribution of nodes was done in the efforts of increasing the uptime of the system (if one node dies, the other node can stay alive and service requests); in modern times with blockchain setting, the same is desirable for a different reason: decentralization (no single node or small set of nodes can disrupt the entire system).

For all further discussions, assume there to be N different **nodes** in some distributed system S . We can then denote each node as $n_0, n_1, \dots, n_{N-1}$. A client C communicates with S by communicating with one of the nodes in S , as such each node responds to C as if being a spokesperson of S , i.e. it presents the services to the client according to the "global view" of the distributed system. Each such node $n_i \in N$ maintains a local *append-only* data structure maintaining the history of "actions" / "transactions" it has seen since start of the protocol which should mimic closely the "global view" of the system. We can call it the "local view" of n_i . The content of messages can be arbitrary but are chosen from a message space M , all possible messages.

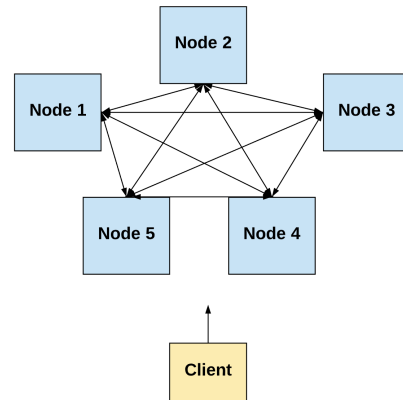


Figure 3.1: An example distributed system with 5 nodes. It may not be necessary that all nodes are connected directly to each other, but all nodes together present themselves as a single unit to server client.

3.1.1 Consistency and Liveness

We begin our analysis of a distributed systems by describing what properties we intend to see from such systems. In very crude terms we want the following two properties:

- **Bad things don't happen / consistency:** We want all nodes to agree on the common history, i.e. no two nodes have a conflict over which transaction happened first. This is especially significant if two transactions t_1 and t_2 both intend to consume a single resource, A write(a, 20) followed by write(a, 40) is not equivalent to write(a, 40) followed by write(a, 20).

- **Good things happen / liveness:** We want *valid* transactions from users to be accepted by the system. No user should be left starving.

Of course, these are strong claims, and we need to quantify the amount of consistency and liveness that we desire. Here are some quantifications for the same:

- **Strong consistency vs Eventual consistency:** Strong consistency presents with some upper bound time guarantee δ after which it is assured that each node in the network will have the right view of some transaction, i.e. if a transaction t was included at time T , then after some $T + \delta$ all nodes have a consistent view of the status of the transaction. Often, a weaker version **eventual consistency** is however more practically feasible though. Eventual consistency provides similar guarantee of global view of transactions, however without some finite time bound δ . This can also be thought of as consistency where $\delta \rightarrow \infty$.
- **Strong liveness vs Eventual liveness:** Strong liveness presents with some upper bound time guarantee δ after which it is assured that client C 's transaction will be included in the *append-only* history of S , i.e. if a transaction t was presented by client C at time T , then after some $T + \delta$, S includes it. Often, a weaker version **eventual liveness** is however more practically feasible though. Eventual liveness provides similar guarantee of liveness, however without some finite time bound δ . This can also be thought of as liveness where $\delta \rightarrow \infty$.

To elaborate on the above, let's take the following example:

A distributed system S consists of 3 nodes $N = \{n_0, n_1, n_2\}$. The client C can interact with any one of the three nodes in N . Assume the local ledgers of all three nodes are empty, as in:

$$\begin{aligned} n_0 : \text{local view} &= [\phi], \text{known unprocessed txs} = \{\} \\ n_1 : \text{local view} &= [\phi], \text{known unprocessed txs} = \{\} \\ n_2 : \text{local view} &= [\phi], \text{known unprocessed txs} = \{\} \end{aligned}$$

Later, C presents transaction t to n_2 , hence the state becomes:

$$\begin{aligned} n_0 : \text{local view} &= [\phi], \text{known unprocessed txs} = \{\} \\ n_1 : \text{local view} &= [\phi], \text{known unprocessed txs} = \{\} \\ n_2 : \text{local view} &= [\phi], \text{known unprocessed txs} = \{t\} \end{aligned}$$

We want the final state of the system, if it is acting as a single unit to settle at the following configuration:

$$\begin{aligned} n_0 : \text{local view} &= [t], \text{known unprocessed txs} = \{\} \\ n_1 : \text{local view} &= [t], \text{known unprocessed txs} = \{\} \\ n_2 : \text{local view} &= [t], \text{known unprocessed txs} = \{\} \end{aligned}$$

Here, the movement of t from "known unprocessed txs" to the "local view" of all nodes in the system constitutes "liveness". However, this happens in stages. First, n_2 moves t from "unprocessed" to "local view". Later, it ask all the other nodes to updates themselves with his local view. This aspect of other nodes updating their local view to match n_2 's local view constitutes "consistency".

3.1.2 Node honesty

In distributed systems, we generally deal with different kind of node faults. We call a node "faulty" when it deviates intentionally or unintentionally from the regular, online, honest participation in the network.

The following are the different kind of faults categorized:

- **Crash fault:** When some node n_i behaves honestly from some origin time t_0 to some arbitrary time t_c , after which it crashes / goes offline. This can be further subdivided into nodes that present a **Fail-stop** failure and nodes that present a **True crash fault**. Nodes that fail-stop halts, notifies everyone of its death and does not respond to any further messages. The nodes that experience true crash fault do not notify on the other hand. Evidently, true crash faults are harder to detect and manage as the rest of the network has to infer through indirect means the halt of the node.
- **Omission fault:** Omission fault is when a node n_i does not present required messages that it should have presented in an ideal setting to the rest of the network. This could be acknowledgements, votes or any other message. It is to be noted that omission of a message may be due to unreliability in the underlying network instead of a malice at the node's end.
- **Byzantine fault:** Byzantine fault is where a node n_i can behave arbitrarily, i.e. we can assume nothing about the node's behavior. Some of the actions exercised by such node could be: behaving like an honest node for some time range, behaving like a crashed node for some other time range, omitting messages, sending deceptive messages in hopes of disrupting the consensus. It should be noted however that we still assume the cryptography is not broken, as breaking that would mean in a system secured by public-key cryptography, the byzantine node can imitate any other node with no restrictions.

3.1.3 Network Synchrony

3.1.4 Consensus

The replication problem in distributed systems could be summarised as follows. Assume a node n_i has a local view $v^* \in V$ with V being the set of all possible local views. A honest n_i would intend to disseminate v^* to all the other nodes.

Consensus in distributed systems is defined as any algorithm A that satisfies the following properties:

- **Termination:** All non-faulty processes eventually decide on some value $v_i \in V$.
- **Agreement:** All non-faulty processes decide on the same value $v \in V$, i.e. $v_i = v \forall n_i \in N$.
- **Validity:** All non-faulty processes decide on a value v^* .

It is important to understand that all the above properties are for "non-faulty" processes, a.k.a. only the honest processes. See 3.1.2 for definition of node faults.

It is also worthwhile noting that aiming for any 2 out of the above 3 properties does not present us with any useful protocol. This can be argued as below:

If we only aim for *termination and agreement*, all the nodes at each step may decide on some hardcoded value $v' \in V$.

If we only aim for *agreement and validity*, all the nodes at each step will try to convince every other node to accept their value v_i and never respond to other's messages. The consensus halts when all nodes are convinced. Unless all nodes started with the same value v , the consensus never halts.

If we only aim for *validity and termination*, all the nodes at each step will decide on their own local v_i .

3.1.5 Permissioning

3.1.6 Global Clock

3.1.7 Public Key Interchange Infrastructure

3.2 State Machine Replication

The **State Machine Replication** problem is to provide a solution that presents a fault-tolerant distributed system where nodes within have the ability to replicate the local state they hold between themselves. The classic examples

here would be database replicas.

To solve for the SMR problem, we need to argue about the following things:

- **Node honesty:** What kind of

3.3 Dolev-Strong: Byzantine-broadcast, SMR, in a synchronous world

3.4 FLP: Can't handle even 1 faulty node in asynchronous world

The domain of distributed systems saw a major breakthrough in 1978 when Leslie Lamport in his paper "[Time, Clocks, and the Ordering of Events in a Distributed System](#)" introduced the concept of logical clocks. Effectively, this paper presented an affirmative answer to the following question:

1978 Leslie Lamport. Can we impose total ordering to events received by different nodes in a distributed system?

As a primer, a partial ordering is an order on a set of elements which satisfies the following properties:

- **Reflexivity:** $a \leq a$ i.e. an element is always ordered with itself
- **Antisymmetry:** If $a \leq b$ and $b \leq a$, then $a = b$, i.e. no two distinct elements precede each other.
- **Transitivity:** If $a \leq b$ and $b \leq c$, then $a \leq c$, i.e. if a precedes b and b precedes c , then a precedes c .

A total ordering is a partial ordering which satisfies the following additional property that each pair of elements is comparable, i.e. for any two elements a and b , either $a \leq b$ or $b \leq a$.

This seemed at the time to be promising for the distributed systems community, as it led to seeming guarantees of consensus over a strict order in distributed systems.

However, a paper that came out in 1985 by Fischer, Lynch and Paterson, titled "[Impossibility of Distributed Consensus with One Faulty Process](#)" showed the following claim is not always true:

1985 Fischer, Lynch, Paterson. Can we always come up with a way of getting computers to agree on a common value in a distributed system, even if some of the computers are faulty?

This paper is often referred to as the **FLP Impossibility Theorem**. Let's understand this further.

3.4.1 The FLP Impossibility Claim

We take the FLP paper's abstract verbatim that reads the following:

FLP Impossibility Theorem abstract. The consensus problem involves an **asynchronous system** of **processes**, some of which may be **unreliable**. The problem is for the **reliable processes to agree** on a binary value. In this paper, it is shown that every protocol for this problem has **the possibility of nontermination**, even with only **one faulty process**.

The key takeaway from this abstract, without key terms being defined is: If we exist in a distributed system where each "process" / node is willing to participate in a consensus protocol, which may take infinite amount of time to reach a consensus, with message **delays being unbounded** and **no shared clock**, with at-most one process being faulty, i.e. dies without notice to others, then it is not always possible to reach to a common global value.

Async model guarantees eventual message delivery Do note that when we say the model of message delivery between processes is **asynchronous**, we mean that there is no upper bound on the time it takes for a message to be delivered. However, the subtle point is that the message **eventually gets delivered and**

is not lost.

Before we establish this however, we need to define some key concepts:

3.4.2 Key concepts

Let's begin with a setting which is not distributed in nature. Consider a client process CL and a server node S . The server S saves a single bit value with itself and presents two distinct operations to the client CL:

- `write(v: 0/1)`: This operation writes the value of a variable given by client at the server S .
- `read() → 0/1`: This operation returns the value of a variable stored at the server S .

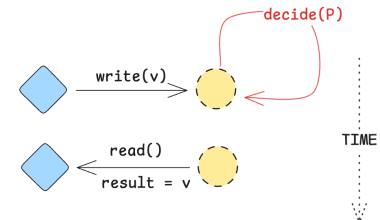


Figure 3.2: The single node case of read and write

A crucial component in this system is existence of a function `decide()`, which can be anything that operates on the internal state of the server. A simplest form of `decide()` could be a function that returns the latest value stored in the server. Another variant could be a function that returns the XOR value of the last 5 writes.

In our case, we can extend the same construction but to a distributed system outwardly presenting itself as a unit but internally being composed of multiple servers called **nodes**. In essence, we desire that the same `write()`, `read()` and `decide()` functions that were present in the single node case, be present in the distributed system as well and **they should act exactly the same**.

The FLP paper imagines the distributed setting to be describable as follows:

There are n nodes in the distributed system. The nodes are connected to each other in connected but not necessarily a complete way in a graph-theoretic sense (a.k.a between any two nodes a path exist but not necessarily of length 1). Each process p_i is defined by $p_i = (I_i, O_i, f_i)$ where I_i is input in the set $\{0, 1\}$, O_i is a one-time write only output in the set $\{\phi, 0, 1\}$, ϕ being the value it exercise if it has not yet been written to, and f_i is a deterministic function that defines the internal state transition algorithm of p_i .

Each process p_i can send a message $m \in M$ (M being the "message universe") to some other process p_x at any time as a tuple (p_x, m) submitted to a globally singleton multi-set **message buffer** (note, source p_i is not described in the tuple).

The **message buffer** is a special non-deterministic (the only component in the system that is not deterministic) entity that presents each process with a function `receive(i) → { $\phi$ ,  $m$ }`. This function when called with index i , either returns ϕ , a null message or m , one of the messages the buffer has received for delivery to process p_i . In the latter case, upon delivering the message, a tuple (p_i, m) is removed from the message buffer. Note how message buffer sufficiently describes an asynchronous message passing system. It can choose to return the null message any number of times, in effect simulating unbounded delay.

In a **synchronous** setting however, we have some upper bound on the time it takes for a message to be delivered. In an **asynchronous** setting, we have no such bound. As such, it is impossible to detect if the counterparty process has failed or is just taking a long time to respond.

One should also discuss a bit about what it means to "fail" in a distributed system. Typically we categorize failures in 4 different categories:

- **Fail-stop**: The process halts, notifies everyone of its death and does not respond to any further messages.
- **Crash**: The process halts, does not notify anyone of its death and does not respond to any further messages.
- **Byzantine**: The process can send arbitrary messages to other processes, including lying about its own state.
- **Permissionless Byzantine**: The process can send arbitrary messages to other processes, including but not limited to impersonating a third-party.

In context of the FLP paper, we are concerned with the second type of failure, the "crash" without notification.

We define a **System Configuration** C as a unique combination of the internal states of all processes and the messages in the message buffer.

3.4.3 Consensus, System Configurations, Events and Schedules

Consensus in distributed systems is defined as any algorithm that satisfies the following properties:

- **Agreement:** All non-faulty processes decide on the same value.
- **Termination:** All non-faulty processes eventually decide on a value.
- **Validity:** All non-faulty processes decide on a value that was some process's internal state at the start.
Corollary: If all non-faulty processes start with the same input value, then the decision value must be the same as the input value.

The system configuration C advances to a new configuration C' when it encounters **events**.

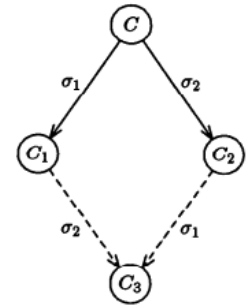
Events are messages (p, m) from the message buffer. Since there are many different messages that can be extracted from C due to message buffer being non-deterministic, many different configurations can stem out from single configuration C . A list of events extracted from C is called a **schedule**.

If at any system configuration C , all schedules lead to a final terminating consensus decision of value v , then we say that the system configuration C is **decisive** / **univalent** for value v . This can also be written as v -valent. Since we are in binary domain, decisive configurations are either 0-valent or 1-valent.

If at any system configuration C , different schedules lead to different consensus decisions, then we say that system configuration C is **indecisive** / **multivalent**. Since we in binary domain, it also is called bi -valent.

3.4.4 Lemma 1: Disjoint Schedules Commute

Suppose that from some configuration C , the schedules σ_1, σ_2 lead to configurations C_1, C_2 respectively. If the sets of processes taking steps in σ_1 and σ_2 , respectively, are disjoint, then σ_2 can be applied to C_1 and σ_1 can be applied to C_2 , and both lead to the same configuration C_3 . See 3.3 for a visual representation.



3.4.5 Lemma 2: There exists a bi-valent initial configuration

Let us assume the contrary to be the case, i.e. all initial system configurations C are decisive, uni-valent. Then, either all are 0-valent, 1-valent or a mix of both with some states being 0-valent and others 1-valent.

A strictly 0-valent or 1-valent configuration is trivial in nature and does not require any consensus protocol to reach a decision. However, a mix of both 0-valent and 1-valent configurations would require a consensus protocol to reach a decision.

Assume n processes participating in the system. Then, all decisive initial configurations are 2^n in number. Since all of them are uni-valent. There would exist some adjacency boundary between 0-valent and 1-valent configurations in the n dimensional lattice, the only process differentiating this being one of the n processes in the system. If such process goes down without notice, the system would be left in a bi-valent state if it intends to resolve the consensus deterministically as it cannot choose 0-valency or 1-valency.

Figure 3.3: Schedules applying to disjoint processes can be out of order

3.4.6 Lemma 3: All roads lead to indecision

Claim: If we start from some bi-valent system configuration C_i , with some specific event $e = (p, m)$ extractable from the message buffer from C_i , there exists a schedule S_{C_i} that leads to a bi-valent configuration C_{i+1} with event e consumed in the process.

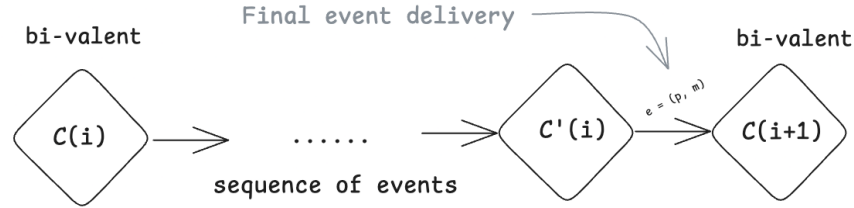


Figure 3.4: A bi-valent configuration can lead to another bi-valent configuration

It is noteworthy that we operate in an asynchronous network model, where-in while it can take a really long time for a message to be delivered, the guarantee exists that it is eventually delivered.

This lemma intends to present that there is a sequence of message deliveries that we can do at each bi-valent configuration that leads to another bi-valent configuration, and also that e getting delivered. Multiple applications of this approach will essentially prove the FLP impossibility theorem.

Let us set-up the proof in the following way:

- Start with a bi-valent configuration C_i .
- Let there be a sequence of events $S_{C_i} = \{e_1, e_2, \dots, e_n\}$, $e_i \neq e \forall i \in [0, n]$ that lead to a bi-valent configuration C_{i+1} . It should be noted that $|S_{C_i}| = 0$ is possible.
- Let C'_i be the "penultimate configuration" that is reached by applying the schedule S_{C_i} to C_i . Since $|S_{C_i}| = 0$ is possible, C'_i can be the same as C_i .

All configurations in 3.4 are bi-valent: It is easy to see that in our definition of system configurations, we can never go from a uni-valent configuration to a bi-valent configuration. In this setup of C_i and C_{i+1} both being bi-valent, all the intermediate configurations including the penultimate configuration C'_i must also be bi-valent.

We now define a few possible configuration categories for intermediate and penultimate configurations shown in 3.4.

- **Pre-uni-valent:** Some configuration C^* is pre-uni-valent, if it is reachable from C_i via some sequence of events without delivering $e = (p, m)$ and application of e to it would transition into a new configuration C^{**} which will be uni-valent. For example, if C' is pre-uni-valent, then C_{i+1} is uni-valent which we do not want.
- **Pre-bi-valent:** Some configuration C^* is pre-bi-valent, if it is reachable from C_i via some sequence of events without delivering $e = (p, m)$ and application of e to it would transition into a new configuration C^{**} which will be bi-valent. For example, if C' is pre-bi-valent, then C_{i+1} is bi-valent which we want.

Note that Pre-X-valency resolving into X-valency for X being uni or bi, is a property of the event $e = (p, m)$.

Note that all Pre-uni-valent configurations necessarily need to be bi-valent. As by definition, at that configuration if a decision is already made, the configuration would reduce to uni-valency. Hence, a Pre-0-valent configuration will have both 0-valent and atleast one of 1-valent and bi-valent configurations as its reachable children configurations. Similar argument goes for Pre-1-valent (atleast one of 0-valent or bi-valent) and Pre-bi-valent (atleast one of 0-valent or 1-valent).

So, what can we say about C_i ? Well, if C_i is Pre-bi-valent, then by definition application of e to it will lead to a bi-valent configuration C_{i+1} . In this case, we have proven bi-valency leading to bi-valency.

The other case however is more interesting. What if C_i is Pre-uni-valent? Well, direct application of e to it will lead to a uni-valent configuration which we want to avoid. We need to path our way from a Pre-uni-valent configuration C_i to some Pre-bi-valent C'_i so that we can transition into a bi-valent C_{i+1} by application of e . **Can we always path a way from a Pre-uni-valent configuration to a Pre-bi-valent configuration?**

We begin with an assumption that C_i is Pre-0-valent. Assume another event $e' = (p', m')$ distinct from $e = (p, m)$ such that application of e' to some reachable configuration X from C_i leads to a non Pre-0-valent configuration Y . See figure 3.5 for a visual representation.

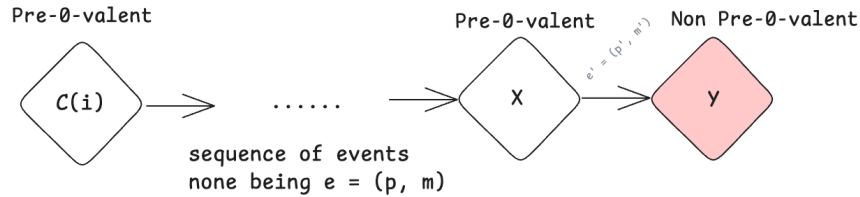


Figure 3.5: Finding a non Pre-0-valent configuration from a Pre-0-valent configuration

We claim to **show that Y is Pre-bi-valent**.

This is because, if Y is Pre-1-valent and e is applied to it, the resulting configuration Y' will be 1-valent. However, since e can be applied to X as well, the resulting X' from X will be 0-valent. If later on, e' is applied to X' , the resulting configuration X'' will also be 0-valent.

However, we note that X'' is identical to Y' by lemma 1 if e and e' are disjoint, i.e. $p' \neq p$. This is a contradiction as Y' is 1-valent and X'' is 0-valent.

On the other hand if $p' = p$, then resolution of deterministic value of choice rests on a single process p . If this process crashes, the system will be left in a bi-valent state.

Hence, under the assumption of possibility of single node failure, transition from Pre-0-valency to Pre-1-valency is impossible. This implies that Y indeed is a Pre-bi-valent configuration.

3.4.7 Proving the theorem

Let us start our protocol from some initial bi-valent configuration C_0 . Existence of such initial configuration is given by lemma 2.

For any transition to C_{i+1} from C_i , we choose the oldest message e_{old} known to be in the message buffer at C_i . Given C_i and e_{old} , we can construct a schedule S_{C_i} that leads to C_{i+1} as per lemma 3 leading to a bi-valent configuration. This also, leads to the consumption of the oldest message e_{old} known at C_i .

We continuously apply the above scheme till the point we are left with no messages in the message buffer. We eventually still reach a bivalent state C_∞ .

3.4.8 Conclusion

- The FLP impossibility theorem makes clear that **trade-offs are required** in the design of distributed systems. Whether it is loosening up on deterministic guarantees or on tightening up on timing guarantees, you cannot have both.
- The above can also be described as, in event of asynchrony, do you compromise on consistency (determinism) or on liveness? **Longest chain (Nakamoto) based consensus protocols like Bitcoin and Ethereum compromise on consistency, while protocols like PBFT compromise on liveness.**

Further Resources

Original Paper : <https://groups.csail.mit.edu/tds/papers/Lynch/jacm85.pdf>

Overview - FLP Impossibility Theorem : <https://www.youtube.com/watch?v=Vmlj-67aymw>

Extending the bi-valent configuration (Lemma 3) : <https://www.youtube.com/watch?v=cwEXmcszbos>

Consize summary : <https://www.youtube.com/watch?v=2fnS-f-mwlo>

3.5 The CAP principle

Chapter 4

Ethereum

4.1 Legacy Ethereum (Ethereum 1.0)

4.1.1 Recursive Length Prefix (RLP) encoding

Recursive Length Prefix (RLP) serialization is used extensively in Ethereum's execution clients. RLP standardizes the transfer of data between nodes in a space-efficient format. The purpose of RLP is to encode arbitrarily nested arrays of binary data, and RLP is the primary encoding method used to serialize objects in Ethereum's execution layer.

RLP does not deal with strings, and other "types" and delegates the encoding of those to the underlying binary data to some higher-order protocols. RLP only deals with binary blobs and positive integers.

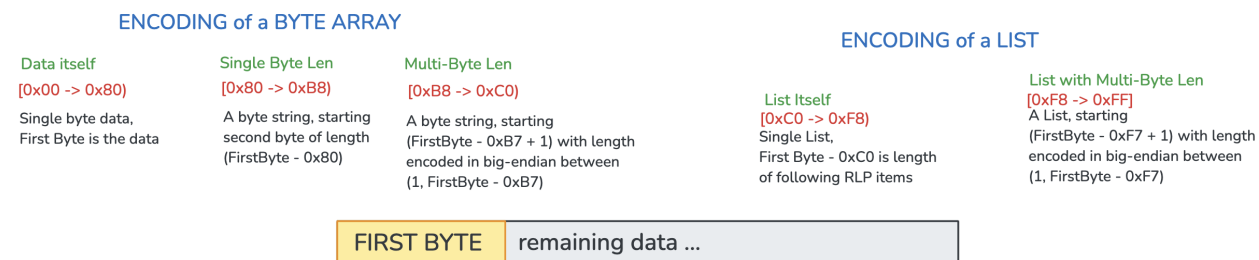


Figure 4.1: The first byte of an RLP serialized blob describes how to parse data.

RLP only deals with "items". Essentially:

- A "byte array" is an item.
- A positive integer is an item. Essentially, this is also converted into a "byte array" by converting it to a smallest possible big-endian representation of the integer.
- A list of items is an item.

The first byte of an RLP serialized blob is a "type" byte ¹. See figure 4.1 for the description.

4.1.2 How to deploy a contract: CREATE vs CREATE2

Deploying a contract on Ethereum involves creating a new contract instance and assigning it an address. The creation part is done via creating an instance of `init_code`, which is a constructor code which when executed

¹RLP on ethereum.org: <https://ethereum.org/en/developers/docs/data-structures-and-encoding/rlp/>

under an EVM call frame, generates the bytecode of the contract. The simplest instance would be a "default constructor" where the `init_code` just returns the required bytecode verbatim.

As far as assignment of the address goes, how that address is assigned depends on the method of deployment. There are two methods of deploying a contract:

- **CREATE**: This is the original method of deploying a contract. The address is determined by the sender's address and the nonce (number of transactions sent from that address). The address is calculated as:

$$\text{address} = \text{keccak256}(\text{rlp}(\text{sender}, \text{nonce}))[12:]$$

where rlp is the recursive length prefix encoding.

- **CREATE2**: Introduced in [EIP-1014](#), this method allows for more flexibility in determining the address. The address is calculated using the sender's address, a salt value, and the bytecode of the contract. The address is calculated using the last 20 bytes of the keccak256 hash of the following:

$$\text{address} = \text{keccak256}(\text{0xff}, \text{sender}, \text{salt}, \text{keccak256}(\text{initCode}))[12:]$$

Addresses generated by `CREATE2` seldom collide with addresses generated via `CREATE`

It is guaranteed that the address will not collide with addresses generated via `CREATE` as the `0xff` RLP prefix is used only for data that is petabytes long. This is because if we compare `rlp(...)` to `0xff + ...`, the `rlp(...)` will seldom have a `0xff` prefix, especially if it is as in `CREATE` opcode (`rlp(sender, nonce)` is too small as compared to petabytes of data that will allow the first byte of RLP encoding to be `0xff`).

Address collision between `CREATE` and `CREATE2`

There still is a small possibility that the address generated by `CREATE2` will collide with an address generated by `CREATE`. When that happens, upon execution, an assert is done which checks that the destination address has any non-zero nonce, and non-zero code. In case that is true, a revert is thrown.

4.1.3 Wintermute Hack (2022): `CREATE` and EIP-155 for transaction signatures

[Wintermute](#) was engaged by the Optimism Foundation for liquidity provisioning services on the \$OP launch. On May 27, 2022, 20 million \$OP was allocated to Wintermute from the Foundation's Partner Fund. However, Wintermute later found that they could not access these tokens because they had provided an Ethereum (L1) multisig address that they had not yet deployed to Optimism (L2), where the allocated funds were sent. Unfortunately, an attacker found this and deploy the multisig contract with the same address to L2 before Wintermute, effectively gaining control over the funds.

Wintermute also later put out a message to optimism community ².

To understand this hack better, let's enumerate the important addresses concerned ³:

1. **MultiSig address(0x4f..81)**: The address 0x4f3a120e72c76c22ae802d129f599bfdbc31cb81 where funds from Optimism Foundation were received on the Optimism chain (Chain ID = 10). This address on Ethereum (Chain ID = 1) hosted a Gnosis Safe Multisig wallet and was controlled fully by Wintermute. This wallet however was not deployed on the Optimism chain and was an unused address.
2. **Gnosis Safe Proxy Factory(0x76..9b)**: The address 0x76e2cfc1f5fa8f6a5b3fc4c8f4788f0116861f9b is the Gnosis Safe Proxy Factory contract deployed on Ethereum, but was not deployed on Optimism. This contract is used to create Gnosis Safe wallets, the MultiSig wallet 0x4f..81 above. This contract used `new` keyword to create the wallet, which uses `CREATE` opcode under the hood.

²Wintermute message: <https://gov.optimism.io/t/message-to-optimism-community-from-wintermute/2595>

³A complete stepwise description of hack has been borrowed from <https://inspexco.medium.com/how-20-million-op-was-stolen-from-the-multisig-wallet-not-yet-owned-by-wintermute-3f6c75db740a>

3. **Gnosis Safe Proxy Deployer**(0x1a..6a): The address 0x1aa7451dd11b8cb16ac089ed7fe05efa00100a6a is the EOA (Externally Owned Account) that deployed the Gnosis Safe Proxy Factory contract on Ethereum. This address did not exist on Optimism however. Since it also used **CREATE** opcode, only it can reliably deploy the same address on Optimism. The attacker did not know the private key behind this EOA, which is a secret that only Gnosis Safe team holds.

Description of the hack

The hack made use of two vulnerabilities:

- Use of the **CREATE** opcode.
- Use of non-chain specific transaction signatures introduced in EIP-155.

The following figure 4.2 shows the steps of the hack.

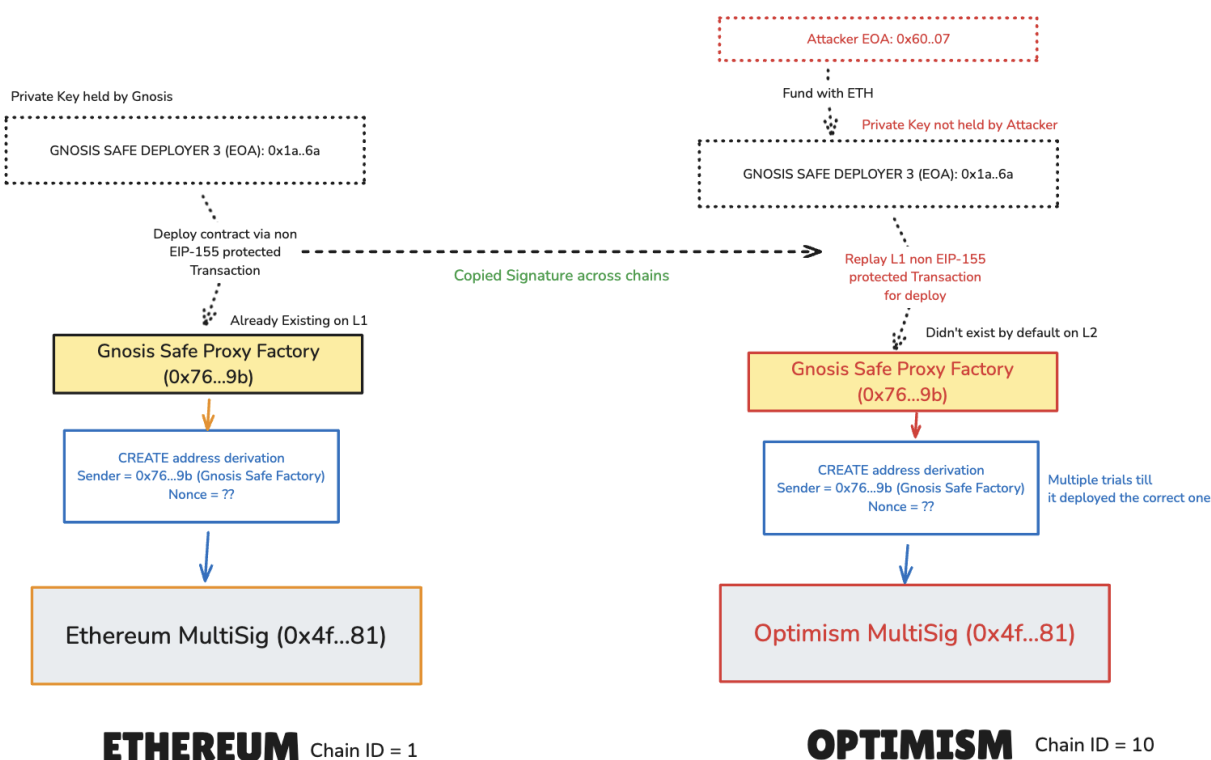


Figure 4.2: The steps of the Winternute hack (2022).

Exercise 1 Ethereum Deployments

1. Explain in detail the difference between **CREATE** and **CREATE2** in terms of how they generate addresses for deployed contracts. Why is 0xFF prefix added when calculating the hash in **CREATE2** ?

4.1.4 Accessing Historical Block Hashes: EIP: 2935

For various applications, it is necessary to access the historical block hashes of the Ethereum blockchain. One such reason is that it provides a good source of randomness. Another good reason is to build an "introspection engine" where contracts on Ethereum can introspect its own history.

Block hashes of the current block can not be known during the context of execution of a transaction. One of the reasons for this is that during execution of a transaction, the block's receipt hash is not built yet, since that in turn depends on all the transactions in the block combined, each transaction cannot have the block-hash.

Ethereum however, has an opcode called `BLOCKHASH` which allows contracts to access the block hash of the 256 most recent blocks. This opcode is useful for applications that require randomness, but it is not enough for applications that require access to the entire history of block hashes.

But what limits the access to historical block hashes more than 256 blocks old? There have been two separate known attempts at solving this. One by Vitalik himself via [EIP-210](#) and another by Vitalik, Tomasz (Nethermind) etc via [EIP-2935](#). As of writing, EIP-210 is marked as stagnant and EIP-2935 is in "Review" state since 4 years.

The primary idea of EIP-2935 was to create a new storage trie that stores the block hashes of all blocks. This would have been located at a special location: `0xffffffff_ffffffff_ffffffff_ffffffff_fffffffe`. This address would essentially be just a state without a bytecode, making it different from all other accounts. Hence, a new opcode `STOP` could

4.2 Ethereum after Merge (Ethereum 2.0)

TODO: Introduce SSZ (simple serialization), a new serialization format for Ethereum 2.0. It is a more efficient serialization format than RLP and is used in the Beacon Chain.

<https://ethereum.org/en/developers/docs/data-structures-and-encoding/ssz/>

https://eth2book.info/altair/part2/building_blocks/ssz/

4.2.1 The Proof-of-Stake Block

In every slot (spaced twelve seconds apart) a validator is randomly selected to be the block proposer. They bundle transactions together, execute them and determine a new 'state'. They wrap this information into a **block** and pass it around to other validators.

Other validators re-execute and validate: Validators who hear about the new block re-execute the transactions to ensure they agree with the proposed change to the global state. Assuming the block is valid, they add it to their own database.

If a validator hears about two conflicting blocks for the same slot they use their fork-choice algorithm to pick the one supported by the most staked ETH.

Previously, the Ethereum's Proof-of-Work Block (PoW Block) only contained execution related information. Post-Merge, the new Proof-of-Stake Block (PoS Block) will contain ACE information:

- Administration (the metadata of the block)
- Consensus (Beacon Chain co-ordination)
- Execution (Block data)

This makes PoW Block more or less a subset of the new PoS Block. See [4.3](#).

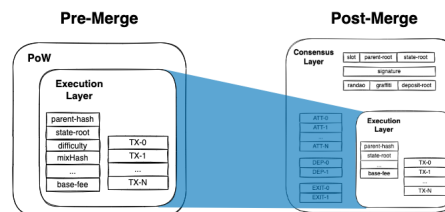


Figure 4.3: The Proof-of-Stake Block

4.2.2 Finality Metalayer: Casper FFG

Talk about:

- Plausible Liveness

- Accountable Safety

4.3 Exercise Solutions

Answer of exercise 1

1. `TODO: Add an answer here`

Chapter 5

Solana

Solana seems to be an interesting trade-off choice in the blockchain trilemma when compared to the likes of Ethereum. The blockchain trilemma is a trade-off between decentralization, security and scalability. Solana seems to be focusing more on the scalability, while security and decentralization are arguably worse as compared to Ethereum.

Very briefly, the blockchain trilemma goes as follows:

- **Decentralization:** The more decentralized a blockchain is, the more secure it is. This is because it is harder for a single entity to take over the network. This is often measured using the *nakamoto coefficient*¹.
- **Security:** The more secure a blockchain is, the more guarantees it can provide, the harder it becomes for an adversary to compromise the system. Guarantees come in the form of economically-backed guarantees (which are subject to elements such as total tokens staked in a network etc) or mathematically-backed guarantees (such as zk proofs, hash non-collision etc.).
- **Scalability:** The more scalable a blockchain is, the more user requests, often measured in form of "transactions per second" or TPS it can perform.

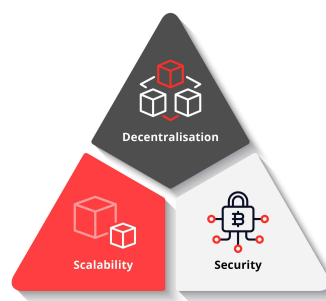


Figure 5.1: The blockchain trilemma

Solana focuses a whole lot more on the "transactions per second" part of the blockchain, a.k.a. the "scalability" aspect.

5.1 The transaction and the block

5.1.1 Transactions and "blockhash queue"

Transaction expiration for long term replay protection

A transaction's recent blockhash acts as a timestamp, and a "blockhash queue" stores the last 150 blockhashes for validation. Transactions are considered recent if their blockhash is within the queue, and if they are not

¹**Nakamoto Coefficient:** First proposed by Balaji Srinivasan and Leland Lee and named in honor of Satoshi Nakamoto, the pseudonymous creator of Bitcoin, the Nakamoto Coefficient is a measure of the smallest number of independent entities that can act collectively to shut down a blockchain. It is often closely related to the "Gini Coefficient" of the economical system under question. For current nakamoto number of public blockchains using Proof-of-Stake, see <https://nakaflo.io/>

processed within a certain timeframe (around 1 minute 19 seconds), they can expire. You can find more details in solana docs here: <https://solana.com/developers/guides/advanced/confirmation#how-does-transaction-expiration-work>.

Transaction Deduplication for short term replay protection

One can argue that while long term replay protection is good, it is not enough. For example, if a transaction is sent to the network and is attempted for execution in two consecutive slots within the 150 slot window, transaction expiration described above will not help. This is where transaction deduplication comes in. The validator maintains a list of transactions that have been executed in the last 150 slots, the "blockhash queue". If a transaction is attempted to be executed again within this window, it will be rejected.

TODO: Talk about durable nonces and how they are used in solana (hardware wallets). These are "long lived transactions" which all txs in ethereum are.

5.2 The Transaction Processing Unit (TPU)

5.3 Proof of History: VDF based transaction ordering

5.4 Entries, Slots, Bank

In Solana, a blockhash is generated at the end of a slot s and represents a cryptographic digest of the bank state at that slot. The exact method of computing it is found in the 'bank.rs' file in the Solana codebase.

The high level idea remains as follows:

$$\text{BlockHash}_s = \text{Hash}(\text{BankHashComponents}_s)$$

If we look deeper into the components of the `BankHashComponents`, we can see that it is a struct that contains the following fields ([code reference](#)):

Listing 5.1: `BankHashComponents` before SIMD-0223

```
1 pub struct BankHashComponents {
2     pub parent_bank_hash: String, // 32 bytes hash of previous bank
3     pub accounts_delta_hash: String, // 32 bytes merkle root hash of accounts deltas
4     pub signature_count: u64, // 8 bytes, number of signatures in this slot
5     pub last_blockhash: String, // 32 bytes, the last blockhash which upcoming transactions will
        refer to
6     pub epoch_accounts_hash: Option<String>, // 32 bytes, the epoch accounts hash
7     pub accounts: AccountsDetails, // Internally, a vector of accounts and details
8 }
```

Let's dig deeper into what each of the above fields mean:

- **Parent bank hash:** The hash of the parent bank, which is the previous block in the chain.
- **Accounts delta hash:** The hash of the accounts that have changed in this slot. This is a merkle tree hash of the accounts that have changed in this slot. Remember, this is not the same as the epoch accounts hash, which is a hash of all accounts at the end of an epoch. Rather, this only presents the changes in the accounts in this slot.
- **Signature count:** The number of signatures in this slot.
- **Last blockhash:** The "last blockhash" that upcoming transactions will refer.

- **Epoch accounts hash:** The merkle hash of the accounts at the end of an epoch. This is populated only at the end of an epoch.
- **Accounts:** The accounts that have changed in this slot and their details.

The transactions' non-replayability is ensured by the `last_blockhash` field. Basically, the transactions encode a reference to the last blockhash, which is the hash of the previous block. This ensures that the transactions are non-replayable and can only be executed once. A transaction for example would be constructed as follows:

Listing 5.2: Example creation of a transaction requiring reference to a recent blockhash

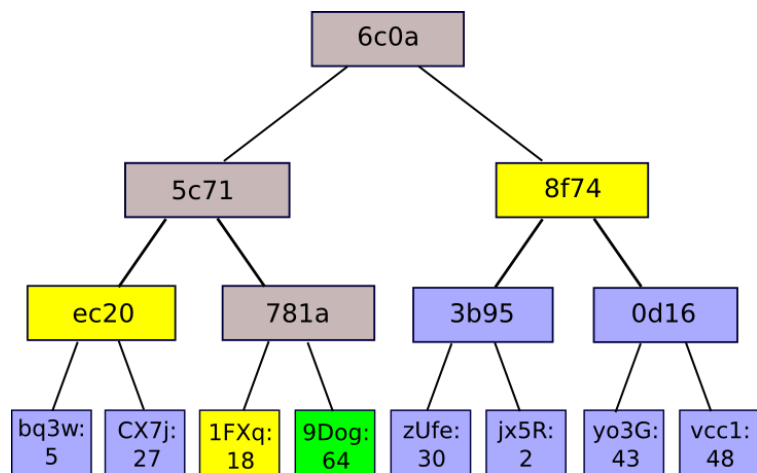
```
1 let tx = system_transaction::transfer(mint_keypair, &dest_keypair, 1, bank.last_blockhash());
```

5.5 State proofs in Solana

State proofs are a necessary component of any blockchain to prove the existence or non-existence of a state object in the system. More generally, state proofs operate as follows:

- **State commitment:** A state commitment C_t is a succinct representation of the entire state of the system at a given point in time. This is used to ensure that the state has not been tampered with and can be used to verify the integrity of the state later using proofs π (described below). Typically the state commitment is generated during the block-building time and is included in the block header. This block header is then signed by the block-builder and a supermajority of the validators in the system, effectively grounding its authority as the "source of truth".
- **State proof:** A state proof is a cryptographic proof that allows a user to verify the existence or non-existence of a state object in the system. This is typically done by creating a proof π for a given key or a set of keys in the state. This proof later is verifiable against the state root.
- **Proof verification:** State proof verification is the process of verifying the state proof using the state commitment.

A classic example of state proofs is the Merkle tree. A Merkle tree is a binary tree where each leaf node is a hash of a data block and each non-leaf node is a hash of its children. The root of the tree is called the Merkle root and serves as the state commitment. The proof for a given key is the path from the leaf node to the root, which can be used to verify the existence or non-existence of the key in the state, see figure 5.2.



5.5.1 Homomorphic state commitment: AccountsLatticeHash

Solana with two improvement documents: [SIMD-0215](#) and [SIMD-0223](#) proposes a new state commitment scheme called *AccountsLatticeHash* and removal of previously existing *AccountsDeltaHash* scheme.

More specifically, while SIMD-0215 proposes a new state commitment scheme

Figure 5.2: A example merkle proof: `6c0a` is the state commitment, the topmost node of the binary merkle tree with 8 leaves. The proof for the key `9Dog: 64` is the path from the leaf node to the root, hence `[1FXq: 18, ec20, 8f74]` are additional data (state proof) against state commitment (`6c0a`) to prove existence of `9Dog: 64`.

(AccountsLatticeHash), SIMD-0223 proposes to remove the old scheme (AccountsDeltaHash).

Before introduction of the AccountsLatticeHash, solana used to have two main kinds of account hashes / state commitments:

- The **Epoch Accounts Hash (EAH)**: This is a merkle tree hash of the full account data, sorted by public key, at the end of an "epoch" in solana.
- The **Accounts Delta Hash (ADH)**: This is a merkle tree hash of accounts, sorted by public key, written in a single block.

The epoch accounts hash is hash of a complete merkle tree and hence can be compared to a "full" state commitment, one against which a state proof for any key can be generated and verified. Ideally, this kind of "full" state commitment is what we want per block in any blockchain. However, generating this is quite expensive in terms of time and compute, leading to this requiring a time greater than 400ms, the average block generation time.

The Lattice Hash Accumulator

The Lattice Hash paradigm shifts from Merkle trees to a homomorphic hashing scheme. Each account a in the global state S is assigned a unique cryptographic hash, denoted $LtHash(D_a)$, where D_a represents the account's complete data relevant for hashing (detailed below).

Each slot in the Solana blockchain has a unique AccountsLatticeHash represented by:

$$ALH = \sum_{a \in S} LtHash(D_a)$$

as such

Each account a in the global state S follows the following structure:

Listing 5.3: Account Structure in solana

```
1 pub struct Account {
2   pub account: Pubkey, // 32 bytes, the key of the account in the state
3   pub lamports: u64,   // 8 bytes, the number of lamports (tokens) in the account
4   pub rent_epoch: Epoch, // 8 bytes, the last epoch in which rent was paid for the account
5   pub data: Vec<u8>,   // variable length, the data associated with the account, could be
                        // program, could be empty etc
6   pub owner: Pubkey,   // 32 bytes, The public key of the program that owns the account
7   pub executable: bool, // 1 byte, whether the account is executable or not
8 }
```

TODO: Discuss about rent in solana

Collectively, the metadata (excluding the variable "data") amounts to 81 bytes per account. Thus, for an account with a 200-byte data field, the total input D_a for its $LtHash$ computation would be 281 bytes.

The Solana blockchain typically sees around 10^2 to 10^3 accounts updates per slot. What this translates to in terms of AccountsLatticeHash is that the hash of the accounts in the next slot is computed as:

$$ALH_{\text{new}} = ALH_{\text{old}} - \sum_{a \in S_m} LtHash(D_{a,\text{old}}) + \sum_{a \in S_m} LtHash(D_{a,\text{new}})$$

where S_m is the set of accounts modified in the new slot. We assume $|S| \approx 10^3$ as of writing this document (22 May, 2025).

5.5.2 Post-SIMD-223: BankHash composition

Since SIMD 0223 remove the AccountsDeltaHash scheme, the BankHash now has:

Listing 5.4: Account Structure in solana

```
1 pub struct BankHashComponents {
2     pub parent_bank_hash: Hash,
3     // pub accounts_delta_hash: Option<String>, // removed in SIMD-0223
4     pub signature_count: u64,
5     pub last_blockhash: Hash,
6     pub epoch_accounts_hash: Option<String>,
7     pub accounts_lt_hash_checksum: Option<String>,
8     pub accounts: AccountsDetails,
9 }
10 pub struct BankHash {
11     pub parent_hash: Hash,           // 32 bytes, the hash of the parent bank
12     pub accounts_lt_hash: ALH,       // 2048-byte, the accounts lattice hash at given slot
13     pub signature_count: u64,        // 8 bytes, the number of signatures in the bank
14     pub last_blockhash: Hash,        // 8 bytes, the number of transactions in the bank
15 }
```


Chapter 6

General Concepts in Adversarial Distributed Systems

6.1 Proof of Stake: The Nothing at Stake problem

At a first glance, Proof of Stake seems to be a great idea. It is more energy efficient than Proof of Work, and it is more democratic in the sense that the more tokens you have, the more say you have in the network.

However, it is not quite comparable to Proof of Work. To understand why, we need to notice the following two properties of PoW:

- **Finite prowess:** In proof of work systems, the prowess (often calculated as the number of puzzles a miner can try) of a miner is limited by the amount of computational power they have. Whether they use COTS, ASICs, FPGAs, all are limited inherently. Whenever a miner sees a fork in the chain, they have a choice to make: which chain to mine on. Should they dedicate their prowess to the fork #1 or should they dedicate their prowess to fork #2? Or should they mine on both equally? Regardless of their choice of prowess distribution, their total prowess is limited.
- **Wrong choice is costly:** If a miner chooses to mine on the wrong fork, they lose out on the rewards they could have gotten had they mined on the right fork. This is because the right fork will have more blocks, and hence other miners due to longest chain fork-choice rule will focus their effort onto it, making it "the canonical chain" in due time, leaving the wrong fork blocks to be wasted effort / orphaned.

The above two properties in Proof-of-Work chains, coupled with the assumption that each miner will make their own choice of fork-selection independently, make them inherently a good canonicalization machine, i.e. given enough time all forks will converge to a single chain.

However, in Proof of Stake, the above two properties are not true. The prowess of a validator is not limited by computational power, but by the amount of tokens they have staked. Any validator can freely attest on both the forks. Secondly, since they are not making a choice here, there is no concept of "wrong choice" being costly. This leads to a problem called "Nothing at Stake".

6.1.1 No penalties = No canonicalization

In many early (all chain-based) proof of stake algorithms, including [Bitcoin](#), there are only rewards for producing blocks, and **no penalties**. This has the unfortunate consequence that, in the case that there are multiple competing forks, it is in a validator's incentive to try to make blocks on top of every fork at once, just to be sure.

The result is that if all actors are narrowly economically rational, then even if there are no attackers, a blockchain may never reach consensus on a "canonical state", and all validators will keep building on both forks. [If there is](#)

an attacker, then the attacker need to only overpower the altruistic nodes (who would exclusively stake on the original chain), and not rational nodes (who would stake on both the original chain and the attacker's chain), in contrast to proof of work, where the attacker must overpower both altruists and rational nodes.

6.1.2 Example: The 1% attacker

TODO: Show how a person controlling >0% stake can canonicalize the wrong fork and thus you cannot trust any fork until its made canonical

6.1.3 The Tragedy of the Commons

The Tragedy of the Commons is a concept which states that if many people enjoy unfettered access to a finite, valuable resource, such as a pasture, they will tend to overuse it and may end up destroying its value altogether. Even if some users exercised voluntary restraint, the other users would merely replace them, the predictable result being a "tragedy" for all.

In the context of Proof of Stake, each individual stakeholder might only have a 1% chance of being "pivotal" (i.e. being in a situation where if they participate in an attack then it succeeds and if they do not participate it fails), and so the bribe needed to convince them personally to join an attack would be only 1% of the size of their deposit; hence, the required combined bribe would be only 0.5-1% of the total sum of all deposits. Additionally, this argument implies that any zero-chance-of-failure situation is not a stable equilibrium, as if the chance of failure is zero then everyone has a 0% chance of being pivotal.

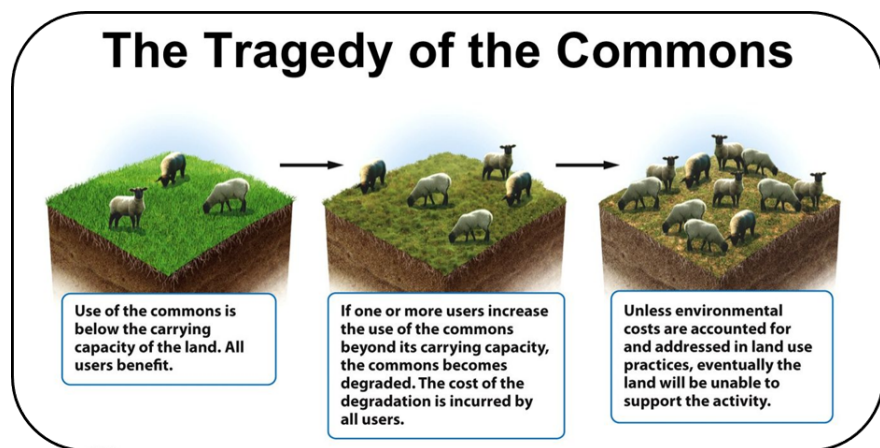


Figure 6.1: Tragedy of the commons

6.1.4 Simulating the "wrong choice is costly"

There are two ways in which one can try solving the "Nothing at Stake" problem, both of which involve making the "wrong choice" costly:

- **Slasher:** In the Slasher algorithm, if a validator is caught signing two conflicting blocks on two different forks, they lose their part / entire stake. TODO: requires elaboration, see why they call it too harsh
- **Casper FFG:** Every validator is forced to choose a fork to validate on. If there are two competing chains, A and B, then if a validator creates a block on B, they get a reward of +R on B, but the block header can be included into A (in Casper this is called a "dunkle" or *proof of malfeasance*) and on A the validator suffers a penalty of -F (possibly $F = R$). TODO: show in Casper design how this is done

6.2 Wealth Inequality and the Gini Coefficient

Many popular blockchains follow a Proof-of-Stake model of consensus. In PoS, the more tokens you have staked, the more faith you present to the network. In return, the network rewards you with more tokens for protocol validation services you provide to the network.

This mainly means that a certain actor with more tokens can stake more tokens, and assuming does follow protocol right, earn more tokens as a result. This is a positive feedback loop, and is often referred to as the "rich get richer" problem. This is a problem because it leads to wealth inequality in the network.

TODO: talk about lorenz curve and gini coefficient

6.3 Miner's Extractable Value (MEV)

Miner's Extractable Value (MEV) is a term coined by Phil Daian et al. in their [Flash Boys 2.0](#) paper. It refers to the value that miners can extract from the transactions they include in a block. MEV is a result of the fact that miners have the ability to order transactions in a block and can choose which transactions to include or exclude. This gives them the power to front-run transactions, re-order transactions, and censor transactions.

Sometimes MEV is also described as "maximal extractable value".

General purpose blockchains provide an almost turing-complete environment¹ for executing smart contracts / programs. Smart contract / programs are sequence of steps or rules that can be deployed at will. Some of these major programs are Decentralized Exchanges (DEX), Automated Market Makers (AMM), and lending protocols. These programs are often used to trade assets, provide liquidity, and borrow assets.

6.3.1 Basic on-chain DeX based arbitrage

If an arbitrage opportunity exists of the following format:

- **Transaction 1:** Decentralized exchange $D1$ is selling asset A in exchange of B at rate $x : 1$
- **Transaction 2:** Decentralized exchange $D2$ is buying asset A and giving B at rate $y : 1$
- **Arb Condition:** $x > y$, i.e. $D1$ gives more A per unit of B than $D2$ needs to exchange for a unit of B

6.4 The ever growing state, state expiry

State expiry refers to removing state from individual nodes if it hasn't been accessed recently. There are several ways this could be implemented, including:

Expire by rent: charging "rent" to accounts and expiring them when their rent reaches zero. This design is implemented in Solana.

Expire by time: making accounts inactive if there are no reading/writing to that account for some amount of time

6.5 Rebuilding the chain / Chain Derivation

6.6 Supporting a different virtual machine

Execution of code in any blockchain assumes an "execution environment". This execution environment is often called the "virtual machine" that blockchain is running. For example, Ethereum runs the EVM, the **Ethereum Virtual Machine**. Solana runs the SVM similarly.

A curious question arises: **Can a virtual machine non-native to a blockchain be introduced as a co-execution environment?**. The answer to this is a resounding yes. However, one needs to question the intent of doing this. Almost all execution environments are [turing complete](#), however the cost of executions may be prohibitive at times.

¹Almost in the sense that execution is still bounded to some maximum number of steps, called gas limit in Ethereum and compute limit in Solana

One such use-case is extending the EVM with capabilities to run WASM or RISC-V code. A couple interesting projects doing this would be [Arbitrum Stylus](#) and [R55](#).

Arbitrum Stylus

Stylus extends the execution environment to run WASM-targeted smart contracts. Instead of writing in solidity, one writes the code in languages like Rust, Zig, Go, Sway, Swift. Rust is supported as primary language and [stylus-sdk](#) is used to build such programs.

R55-Eth



Figure 6.2: Arbitrum Nitro

Chapter 7

Layer 2 Scaling Solutions

7.1 General classification

We classify Layer 2 scaling solution according to their setup:

- **Plasma:** A tree of sidechains (chains of chains) where each user is asked to ensure the part of state it owns is available. Here, significant interaction with L1 is only required in case of dispute resolution. This came much before other solutions (proposed by Joseph Poon and Vitalik Buterin in 2017) but it was too complex to build a useful plasma.
- **Rollup:** An ecosystem running in parallel to a Layer 1 solution that has its own state transition function (STF). After a bunch of transactions and blocks are created, the rollup posts transaction data / state-diff on chain and proves the combined state transition via optimistic or succinct zk proof.
- **Validium:** Similar to a rollup, just that the data-availability of the transaction is kept not on L1, but a separate *data-availability committee* or the validium operator themselves. Validium saves on recurring cost to make data available on L1, but at the cost of security.

7.2 Where sits the enormous data?

Whenever a layer 2 scaling solution is built, the layer 1 contract is provided with the root commitment of the updated state. But a commitment is useless unless it can be opened. A *witness* links a leaf to the commitment and it requires a view of the state / transaction data for things like disputes and withdrawals.

Where does this data exist for generating a witness to do anything useful? A lot of answers can come up:

- Let the data sit on the Layer 1 itself, the rollup route. This is the simplest of the solutions but seldom the most practical. Layer 1s are expensive to post data to.
- Let the data sit on some other special data-availability layer which has some presence on the Layer 1 concerned. For example, posting on celestia. This is the validium route. Here you still pay to post on Celestia albeit much less.
- Let the data sit on a data-availability layer even cheaper than celestia, maybe with a smaller quorum. This is an even cheaper validium route. But the trade-off is security, as the data-availability layer may not be as secure as for example celestia.
- Let the data sit with the users themselves. Let them maintain the data and post it when required. This is the plasma route. This is the most decentralized of the solutions but also the most complex to build. Also, it requires the users to be online all the time. A pretty bad UX.

7.3 Plasma: Each one save one

The setup of Plasma entails that if each part of the state has a well-defined owner, then the owner can be made responsible for keeping their part of the system available.

Essentially, all Plasma chains have an **Operator** who keeps the chain moving. The system is setup optimistically with an intrinsic level of trust on this operator. If the operator misbehaves, it remains the responsibility of the users to exit the chain.

Plasma chains are multi-layered chains where every "Parent Chain" settles for its child chains. The topmost "Parent Chain", the root chain settles on a well-known settlement platform and calls it the Layer 1. Each parent chain is responsible for three basic events through its contract:

- **New Deposit:** Whenever anyone submits a new token deposit of the form (depositAddress, amount, blockHeight).
- **Block Progression:** Only submit the root-hash of the block, *not the data contained within it*. The data within hash to be kept in a good condition (and available) by the user. This takes the form (blockHeight, rootHash).
- **New Withdrawal:** If the operator misbehaves, the users are supposed to exit the plasma chain. This request is posted directly on Layer 1, as it may be assumed that the operator is not available / misbehaving. This takes the form (withdrawaladdress, amount, blockheight, txIndex, outputIndex).

7.3.1 Operator misbehaviors and exits

7.3.2 Plasma Cash

In **Plasma Cash**, since users are only keeping track of their own tokens, they don't know who owns any of the other tokens. When a user wants to send their token to another user, they need to prove that they actually own that token! This creates a problem where each user is expected to be *perennially online* to ensure they can generate a witness against the current state-root of plasma chain.

In such a setup however, other inefficiencies also arise, because of the non-fungible nature of the tokens involved. In a ledger model of accounting, spending 0.1 Token or spending 20 Token out of a 50 Token balance are the same complexity-wise. However, the same is much harder with NFTs given we have to establish the denominations beforehand for all possible transfers in the future, or deal with splitting and merging of tokens.

7.4 Bottlenecked by DA

Layer 2 rollups have been around for a while. The first Layer 2 scaling solution was Plasma, proposed by Joseph Poon and Vitalik Buterin in 2017. The idea was to create a tree of sidechains, each of which would be able to process transactions in parallel. The root chain would be responsible for the security of the system, while the sidechains would be responsible for processing transactions.

Rollups however needs to guarantee that the state of rollup is **data-available**. Otherwise, if state is not available and the rollup-operator stops working, users cannot withdraw. As, even though the root commitment to the entire state is available, the opening are not. This makes rollups constrained by data-availability capacity of the Layer 1 chain.

Ethereum provides 375 KB blobs every 12 seconds.

The following question hence often comes up: *Can we eliminate need for state availability in L2 scalability solutions?*

7.5 Rollups and levels of confirmation

7.6 Faster Layer 2 Interoperability

In late 2020, Ethereum community started focusing on [rollup-centric roadmap](#). The idea of this was to enable various Layer 2 scaling solutions that may potentially provide a whole lot more of execution capacity than the Ethereum (henceforth to be referred as Layer1 or L1) will be capable of.

However, Layer 2 come with following new concerns:

- **Security:** The security of Layer 2's is the concern of how easy it is to compromise a L2 in comparison to an L1. Generally, in a proof-of-stake system like ethereum, a lot of security is borrowed from stake at play. As of writing, this is around 33.8 million ETH staked / around 136.8 billion USD securing the ecosystem. Can the L2 match this level of security?
- **Decentralization:** Control of the system especially with chains like Ethereum is fairly decentralized. As of writing, around 1 million validators run the nodes for the main chain¹. Layer 2 solutions hardly have decentralized sequencers and hence sit almost on the far end of decentralization argument. Can the L2 match this level of decentralized controls?
- **Fragmentation:** Every L2 is almost like a separate "room", although situated within the same "house", the L1. This eventually leads to fragmentation of both users, and the liquidity. On a DEX, liquidity is directly tied to the quantity of tokens users have committed to the liquidity pools. If a crypto asset lacks sufficient liquidity, token holders may face difficulty selling their tokens when they wish. L2 brings in another dimension to this problem. Not only are tokens spread thin among various DeFi apps on the same chain, they are now spread among various DeFi apps on *various* chains.

The Interoperability problems is concerned with the *fragmentation* issue brought in by the advent of L2s.

In its most basic essence, Interoperability is concerned with a transaction that intends to affect state of two different L2 rollup chains. For example, a token transfer from *A*'s account on rollup *X* to *B*'s account on rollup *Y* is a transaction that affects the state of both *X* and *Y*.

¹This number is a bit misleading given a lot is contributed by single institutions like [stake.fish](#) running multiple validator nodes given each validator can only stake 32 ETH per validator. [The Ethereum Pectra Upgrade \(EIP-7251\)](#) is supposed to fix it with increased MAX_EFFECTIVE_BALANCE the staking balance per validator, in hopes consolidating many such large validators into fewer entities

Chapter 8

An on-chain system for asynchronous transactions

8.1 Definitions

We begin the description of our system by defining a few important terms.

An **user**, denoted using U_i , is an entity that interacts with the system to register their asynchronous **intent of execution** ψ_j . A user can be a person, a smart contract, or a machine. The subscripts i and j are used to distinguish between different users and intents, respectively and one user U_i can have more than one intent registered at the same time.

Consequently, each ψ_j has a corresponding intent identifier j and has a unique user i who has registered it for asynchronous execution.

Each intent of execution ψ_j is composed of a list of **actions** $\{\psi_{j,0} \dots \psi_{j,n}\}$ that are to be executed in the specified order within a given **execution opportunity window (EOW)**. The EOW is a time period during which the actions of the intent can be executed by the system and is specific to an intent, progressing the intent to its completion and yielding fee for the solvers involved. However, the user may want to describe a condition (such as a certain period of time after the first action was executed, the "time to live") under which the intent itself should be *collapsed*, in effect making it impossible to progress further.

Listing 8.1: Intent of Execution

```
1 struct Intent {
2     intent_identifier: u64,
3     intent_owner:     Address,
4     actions:          Vec<Action>, // count should be available using 'actions.len()'
5     isEowCollapsed:   Fn(&self) -> bool,
6 }
```

Listing 8.2: Action

```
1 struct Action {
2     action_identifier: u64,
3     intra_dependencies: Vec<(u64, u64)>, // list of intra-intent dependencies
4     inter_dependencies: Vec<(u64, u64)>, // list of inter-intent dependencies
5     accessed_addresses: Vec<Address>, // list of dApps and addresses accessed by the action
6 }
```

8.2 Time Turner

The time turner is a smart contract or a set of smart contracts that allows users to register their intents of execution. The time turner is responsible for managing the registration of intents and ensuring their correct execution. It is also responsible for managing the state of the system and ensuring that all intents are executed in a consistent manner. Also, time turner provides a mechanisms against different actor's misbehaviors and penalizes according to the "asynchronous execution protocol" (AEP) rules.

Time turner maintains a mapping of all registered intents ψ_j to their corresponding user identifiers U_i , called the **intent-to-user-map** which exposes the following function: `getIntentOwner(j) → i`.

8.3 Asynchronous Execution Protocol (AEP)

This section describes the Asynchronous Execution Protocol (AEP) that is used to execute the intents of execution registered by users. We first describe the different components of the AEP and then describe the full flow of the protocol at the end.

8.3.1 Inter-Intent Dependencies

An action $\psi_{j,k}$ can conditionally depend on the successful execution of one or more actions from different intent than ψ_j : $\psi_{x,y}, \psi_{a,b}, \dots$ under intents that are registered by the same user or a different user. This is done by specifying a set of **inter-intent dependencies** $\{\text{INTER}_{j,k} \dots \text{INTER}_{j,n}\}$ for each action in the intent. The inter-intent dependencies are used to ensure that the actions of the intent are executed in the correct order and that the dependencies are satisfied before executing the action.

Each inter-intent dependency $\text{INTER}_{j,k}$ is essentially a list of action references in form of a tuple (x, y) where x is the intent identifier and y is the action identifier within the intent x .

For example, if some action $\psi_{13,2}$ of intent ψ_{13} depends on the successful execution of action $\psi_{12,5}$ of intent ψ_{12} and the successful execution of action $\psi_{7,1}$ of intent ψ_7 , then the inter-intent dependency is represented as $\text{INTER}_{13,2} = [(12, 5), (7, 1)]$. Doing this allows $\psi_{13,2}$ to be executed only after some successful execution of both $\psi_{12,5}$ and $\psi_{7,1}$ and during the execution of $\psi_{13,2}$, the time turner will provide the context object generated by the execution of $\psi_{12,5}$ and $\psi_{7,1}$ to the execution of $\psi_{13,2}$.

Two forms of dependencies are possible:

- **hard dependency**: $\psi_{j,k}$ executes only if all the dependencies are executed and the intent they are part of has transitioned into the status of COMPLETED.
- **soft dependencies**: $\psi_{j,k}$ executes only if all the dependencies are executed, without any care of the status of the intent they are part of. These dependencies while executing faster than hard dependencies, are subject to the risk of consuming a wrong action context object (since the action they depend on may have executed differently post execution forking) or even worse, the intent may have failed, leading to all side effects being rolled back and ADRs not being redeemed eventually.

TODO: Note: Side channel context injection is possible via solver's runtime inputs. However not enforced by the time turner.

8.3.2 Intra-Intent Dependencies

As a counternote to "Inter-Intent Dependencies", we also have "Intra-Intent Dependencies". These are dependencies that exist within the same intent. For example, if action $\psi_{j,k}$ depends on the successful execution of action $\psi_{j,m}$, then the intra-intent dependency is represented as $\text{INTRA}_{j,k} = (j, m)$. This is used to ensure that the actions of the intent are executed in the correct order and can potentially be parallelized if they do not depend on each other.

It is worth noting that Intra-Intent dependencies are not strictly soft-dependencies only. Hard dependency under intra-intent regime would lead to a deadlock situation, the action cannot execute until the whole intent is executed

and the intent cannot be fully executed until the constituent actions are all executed. Hence, the time turner does not allow hard dependencies under intra-intent regime.

8.3.3 Anatomy of an Action

Each actions $\psi_{j,k}$ of an intent ψ_j is composed of three components:

- **Action Pre-condition:** A unique identifier for the action within the intent. It is denoted as k .
- **Action Execution:** The entity responsible for executing the action. It is denoted as $S_{j,k}$.
- **Action Context update:** The receipt generated by the action solver upon successful execution of the action. It is denoted as $\Delta_{j,k}$.

8.3.4 Execution Forking for optimal fee

Assume an intent ψ_j is registered by user U_i at time t_0 and is composed of N actions internally, $\{\psi_{j,0}, \psi_{j,1}, \dots, \psi_{j,N-1}\}$.

After some time t_1 (where $t_1 > t_0$), the time turner is able to find for the contiguous range $\{0 \leq k < K; K < (N-1)\}$ the solvers $S_{j,k}$ that have executed the actions $\psi_{j,k}$ for a fee of $F_{j,k}$. As such, the cumulative fee for executing the actions $\{\psi_{j,0}, \psi_{j,1}, \dots, \psi_{j,K}\}$ is a sum of solver fees and dApp option premiums:

$$F_{Partial} = \sum_{k=0}^{K-1} F_{j,k} + \sum_{k=0}^{K-1} \sum_{l=0}^{ops(j,k)} P_{j,k,l}$$

We can denote this partial execution using $E_{j,k} = (S, F_{Partial})$.

At this point, if a different set of solvers $S'_{j,0}, S'_{j,1}, \dots, S'_{j,K}$ are able to execute the actions $\psi_{j,0}, \psi_{j,1}, \dots, \psi_{j,K-1}$ for a fee of $F'_{j,0}, F'_{j,1}, \dots, F'_{j,K-1}$. Similar to above, the cumulative fee for executing the actions $\{\psi_{j,0}, \psi_{j,1}, \dots, \psi_{j,K-1}\}$ is a sum of solver fees and dApp option premiums:

$$F'_{Partial} = \sum_{k=0}^{K-1} F'_{j,k} + \sum_{k=0}^{K-1} \sum_{l=0}^{ops(j,k)} P'_{j,k,l}$$

We can denote this partial execution using $E'_{j,k} = (S', F'_{Partial})$.

Execution forking: We say execution E' is preferred over E if one can use lesser total fee for executing the chain $[0, K)$, i.e. $F'_{Partial} < F_{Partial} - R$ where R is the sum of all partial refund of premium one can get upon surrendering the execution's dApp options in E , the **premium refund** described below.

This rule does not stand just for partial execution, but also for the entire execution of the intent.

8.3.5 DApps' "Forking Loss" compensation

The dApps involved in the actions of the intent are also supposed to be compensated for the loss of fee due to execution forking. To understand the necessity of this, consider an action reserving the "Option" from a ETH-USDC Swap DApp for "0.5 ETH to 1500 USDC swap" at a later max time of redemption t_{redeem} . Holding this option for an intent is like holding an option in traditional finance.

Since the premium the dApp charges is charged for holding the option till time t_{redeem} , the question arises that what happens if the option is confirmed to be not exercised / surrendered before t_{redeem} by the time turner? Option surrendering before redemption time can lead to dApps being able to take early decisions and provide better pricing for execution of other actions and hence, the dApp may choose to pay back some portion of the premium to the user. This is called **premium refund** and is denoted by $R_{j,k,l}$. This is entirely optional and upto dApps to implement however. There is a valid case where the dApp may not want to refund the premium at all.

8.3.6 Intent grieving by solvers

Every execution forking incurs a cost to the user in form of a "forking loss". This is attributable to the option holding cost on the fork not chosen. This can be exploited by malicious solvers to grief the user by executing the intent in a way that is not optimal for the user.

More precisely, assume an intent ψ_j executed without any forking in its lifecycle. This intent's execution can make full use of ψ_{MAXFEE} . However if we compare this to a case where the intent had some partial execution but then forked to this execution, the user would have to pay an additional fee of holding the dApp options for the previous forked execution, and part of the ψ_{MAXFEE} is lost in paying for options never redeemed.

The way in which a solver can profit from this mechanism is if it colluded with the dApps such that the dApp options are guaranteed to not be redeemed due to a pre-meditated post-hoc execution forking of an intent post option buy from the dApp. Then, a malicious solver can execute the intent partially, make the intent pay for the dApp's option and later fork the execution into a better path, making the dApp pocket the option premium minus premium refund. This profit, also called **griefing profit** can then be shared between the solver and the dApp via some internal arrangement.

The mechanism of why this is possible is similar to the case of MEV. This is because the solvers find themselves in a special position to be able to affect the execution of the intent which very few actor apart from them can.

8.3.7 Execution Payments

Each intent ψ_j composed of a set of actions $A_j = \{\psi_{j,0} \dots \psi_{j,n}\}$ has to pay the intent's action executing solvers $S_j = \{S_{j,0} \dots S_{j,n}\}$ and dApps involved in each of the actions a fee for advancing the intent and getting the action delta receipts(ADRs) $\{\Delta_{j,0} \dots \Delta_{j,n}\}$.

The maximum fee amount that the user is willing to pay for the intent over all the constituent actions is called **max intent fee** and is denoted by ψ_{MAXFEE} . It is specified by the user U_i at the time of intent registration. For simplicity of the argument, let us assume that ψ_{MAXFEE} is denominated in the native currency of the blockchain, for example, ETH for Ethereum and SOL for Solana.

However, the payment amount can be denominated in any currency that holds value on-chain including ERC-20 tokens, and a combination of ERC-20 and native tokens. We call the two formats of payment **native fee regime** and **composite fee regime** respectively.

There exists a possibility that the intent upon execution needs more fee than the maximum fee specified by the user at the time of the registration. In such a case, the user may be given an option to either pay the extra fee or cancel the intent (the default behavior in case of a non-response). As such the time turner exposes a function `addMaxIntentFee(j)` to increase the max fee of ψ_j .

8.3.8 Intent Registration

8.3.9 Intent halt due to EOW collapse

8.3.10 On Options default by dApps or solvers

Chapter 9

Golang

Golang things

Chapter 10

Rust

RUSTY things

10.1 Iterators

IMPLEMENT RLE ITERATOR here: <https://leetcode.com/problems/rle-iterator/description/>

Chapter 11

Solidity

11.1 The memory model

11.1.1 Memory types: `calldata` vs `memory` vs `storage`

Working with solidity requires us to think deeply about the memory model. Three data locations exist ¹:

- `calldata`: This is a read-only area where function arguments are stored. It is immutable and cannot be modified.
- `memory`: This is a temporary area where data can be stored and modified and available during the execution of the call. It is erased between function calls and does not persist on the blockchain.
- `storage`: This is a persistent storage area on the blockchain.

11.1.2 The on-chain storage

Each contract can only store data in its own storage ². Ethereum gives each contract 2^{256} slots of storage, where each storage is 32 bytes. Slots are contiguous and are referenced by indexes, starting at 0 and ending at 2^{256} . All slots are initialized to a value of 0. Each contract's storage area has more slots than all the stars in the universe.

Static sized object representation

Every time we declare a struct, it follows the same rules of representation as C / C++ / Rust. For example, the following contract stores the struct as follows:

Listing 11.1: In-storage representation

```
1 contract UnpackedRepresentation {  
2     bool boolVal;    // 1 bit of data, represented in slot 0  
3     uint256 largeVal; // 32 bytes of data, represented in slot 1  
4     uint8 smallVal;  // 1 byte of data, represented in slot 2  
5 }
```

A better alternative of this could have been:

¹Alchemy doc explains with an example: <https://docs.alchemy.com/docs/what-is-the-difference-between-memory-and-calldata-in-solidity>

²Alchemy doc on storage: <https://docs.alchemy.com/docs/smart-contract-storage-layout>

Listing 11.2: In-storage representation(better)

```

1 contract PackedRepresentation {
2     bool boolVal;    // 1 bit of data, represented in slot 0
3     uint8 smallVal;  // 1 byte of data, represented in slot 0 (alongside boolVal)
4     uint256 largeVal; // 32 bytes of data, represented in slot 1
5 }

```

Dynamically sized object representation

Two primary dynamically sized objects exist in solidity: Dynamically sized arrays and Mappings. For each, the following is done:

1. Similar to static sized objects, a slot is chosen for the object, say slot 3, this is the "marker slot" and holds only the length of the array or the mapping.
2. The first element of the array is stored at `keccak256(3)+0`. The second element is stored at `keccak256(3)+1` and so on.
3. In case of mappings, the mapping of some key k , in a mapping whose marker slot is m , is stored at `keccak256(h(k), m)`. The function h is dependent on the type of the key. For example, if the key is a string, then h is the key value unpadded. If the key is an integer, then h is the identity function padded to nearest 32 bytes.

11.2 Function and their modifiers

11.2.1 Visibility Modifiers: `private`, `internal`, `external`, `public`

Solidity has four visibility modifiers for functions³. In increasing order of access they are (mnemonic **PrIEPu**):

- `private`: Accessible by functions in the **same contract**, not even derived contracts.
- `internal`: Accessible by functions in the same contract **and derived contracts**.
- `external`: Accessible by functions from **other contracts**, but not internally (no call from the same contract).
- `public`: Accessible by functions everywhere.

The general rule of thumb should be applied. Give each function the least amount of accessibility as possible.

11.2.2 Function Modifiers for preconditions and postconditions

Solidity provides us with a way to modify the behavior of functions using "function modifiers". Modifiers are often used to add preconditions or postconditions to functions, or to restrict access to certain functions.

The following is an example of a function modifier:

Listing 11.3: Solidity isOwner modifier

```

1 modifier onlyOwner {
2     require(msg.sender == owner);
3     _;
4 }

```

³This youtube video has a good explanation: <https://www.youtube.com/watch?v=KTEp9xfu2tc>

Here, the `onlyOwner` modifier checks if the sender of the message is the owner of the contract. The `_;` statement is a mandatory placeholder for the function body that uses this modifier. A good explanation of this can be found [here from freecodecamp](#).

Listing 11.4: Contract with modifiers

```
1 contract BaseContract{
2     address owner;
3
4     modifier onlyAllowedUser(address user) {
5         require(owner == user);
6         _;
7     }
8 }
9
10 contract DerivedContract is BaseContract {
11     function updateData(
12         bytes32 newData,
13         address user
14     ) onlyAllowedUser(user) public {
15         // function body
16     }
17 }
```

11.2.3 Special modifiers: `view`, `pure` and `payable`

In Solidity, functions can have special modifiers like `pure`, `view`, and `payable` that indicate their behavior, especially regarding how they interact with the blockchain's state or handle Ether.

- `view`: This modifier indicates that the function does not modify the state of the contract. It can read the state but cannot change it. It is used for functions that only return values based on the current state of the contract.
- `pure`: This modifier indicates that the function does not read or modify the state of the contract. It can only use its input parameters and return values. It is used for functions that perform calculations or operations without relying on the contract's state.
- `payable`: This modifier indicates that the function can accept Ether. It allows the function to receive and process Ether sent along with the transaction. It is used for functions that involve financial transactions or require payment.

Exercise 2 Solidity Basics

1. Explain the difference between `calldata`, `memory` and `storage` in Solidity.
2. Assume we deploy the contract at 11.2.2 on the Ethereum blockchain, without any parameters while deploying. What will be the value of `owner` in the contract (BaseContract)?
3. Write a solidity contract that gives you the N-th Fibonacci number.
4. Differentiate between `pure`, `view` and `payable` functions in Solidity.

11.3 The EIP-721 Standard: Non-Fungible Tokens (NFTs)

Modifier can take in arguments as well and can be borrowed from other contracts. Take for example this:

11.4 Exercise Solutions

Answer of exercise 2

1. Refer to [11.1.1](#)
2. The value would be the zero address. `0x0000000000_0000000000_0000000000_0000000000`

Listing 11.5: Fibonacci Contract

```
3.
1 contract FibonacciContract {
2     uint256 f0 = 0;
3     uint256 f1 = 1;
4
5     function getNthFibonacci(
6         uint256 n
7     ) public
8     view
9     returns (uint256) {
10         if (n == 0) {
11             return f0;
12         } else if (n == 1) {
13             return f1;
14         }
15
16         uint256 a = f0;
17         uint256 b = f1;
18
19         for(uint256 i = 1; i < n; i++) {
20             uint256 added = a + b;
21             a = b;
22             b = added;
23         }
24
25         return b;
26     }
27 }
```

4. Refer [11.2.3](#) for the definitions.

Chapter 12

Binary Search

Binary search is a simple technique of problem solving that splits the solution space in half at each step. But when it comes to implementation, it's rather difficult to write a bug-free code in just a few minutes.

A lot of issues arise when implementing binary search such as:

- Off-by-one errors
- Incorrect bounds
- Incorrect mid calculation
- Incorrect condition to update bounds

12.1 Basic: Minimize k such that $f(k)$ is true

This is the most basic form. Basically because the search space is given directly. We would be given a vector $[A_0, A_1, \dots, A_n]$ and we need to find the minimum k such that $f(k)$ is true.

12.2 Advanced: Search spaces with `feasible(x) -> bool`

Finding the search space could be tricky sometimes¹. However, one common element in these problems is the aspect of monotonicity. Here, if $f(k)$ is true, then $f(k + 1)$ is also true. Following are some good examples that establish the concept.

12.2.1 Problem [SeqShip]: Sequential objects shipping within D days

Problem: Assume a sequential line of objects O with their individual weights described in an array that are situated at port A and want to be shipped to port B . Every day, a ship of some maximum weight capacity m makes trip from A to B transporting some objects in O . Given a maximum days D that shipping company can tolerate to transport all objects in O , find lowest possible m . An example object line could look as follows:

$$O = [6, 7, 1, 2, 3, 4, 5, 8, 9, 10]$$

Given $D = 5$, the minimum m would be 15. Because we can ship items as: $[6, 7]$, $[1, 2, 3, 4, 5]$, $[8]$, $[9]$, $[10]$.

Discussion: This problem does not have a clear search space *prima facie*, but the thing we realize is it asks us to minimize m . Now let's run this idea: if some m works for shipping in D days, would $m + 1$ work also? More

¹A good description is available on Leetcode: <https://leetcode.com/discuss/study-guide/786126/Python-Powerful-Ultimate-Binary-Search-Template.-Solved-many-problems>

concretely, assume that a ship of capacity 10 is able to do the work at hand, can a ship of capacity 11 do it too? The answer is yes. We can call this function $f(x)$ more aptly `feasible(x) -> bool`.

But what should be the search space? Well, the minimum we need is atleast $\max(O)$ since we need to ship the heaviest object in some trip, even if it is the only item shipped that day. The maximum we need is probably shipping all objects in one day. So, the search space is $[\max(O), \text{sum}(O)]$.

Solution: The problem can now be reduced to: Find m such that $f(m)$ is true where $f(m)$ is defined as: *Can we simulate shipping O in D days with a ship of capacity m ?* in the search space $[\max(O), \text{sum}(O)]$.

Each simulation is an $O(n)$ operation where $n = |O|$. Maximum simulations is $\log_2(\text{sum}(O) - \max(O))$. Hence runtime is $O(|O| \cdot \log_2(\text{sum}(O) - \max(O)))$.

12.2.2 Problem [KokoBanana]: Monkey eating piles of bananas

Problem: Assume a monkey is hungry and wants to eat some bananas. There are G piles of bananas, each pile i has b_i bananas. The monkey can eat k bananas per hour and has a maximum of D hours to eat all bananas. Each hour, the monkey can decide any one pile and eat k bananas from it, if there are less than k bananas, it eats all but doesn't eat from any other pile that hour. What is the minimum k for which the monkey can eat all bananas in D hours?

Example scenarios:

$$G = 4, \text{Piles} = [3, 6, 7, 11], D = 8 \quad k = 4$$

$$G = 5, \text{Piles} = [30, 11, 23, 4, 20], D = 5 \quad k = 30$$

$$G = 5, \text{Piles} = [30, 11, 23, 4, 20], D = 6 \quad k = 23$$

Discussion: Very similar to problem [SeqShip], this problem also has a monotonicity property. If k bananas per hour works, then $k + 1$ bananas per hour would also work, albeit not the most optimal. This also uses `feasible(x) -> bool` function.

But what is the search space? The minimum k would be 1 as that's the least a monkey has to eat per hour to make progress hour-on-hour. However, we can probably do better than that. The minimum should be no lesser than $m = D / \text{sum}(\text{Piles})$ as that's the least number of bananas the monkey has to eat per hour to eat all bananas in D hours assuming no inefficiencies, i.e. every hour is spent eating exactly m bananas. This is unlikely since not all b_i may be exactly divisible by m .

The maximum k would be $\max(\text{Piles})$ as that's the most bananas the monkey can eat in one hour given that in each hour it can only eat from a single pile.

The search space is hence $[D / \text{sum}(\text{Piles}), \max(\text{Piles})]$.

Also note an interesting property: a monkey cannot eat any faster than in G hours. Hence any problem where $|\text{Piles}| > G$ would have no solution.

12.2.3 Problem [SplitM]: Split array into m subarrays

Problem: Given an array A of non-negative integers, split it into m subarrays such that the maximum sum s of any subarray is minimized. Example:

$$A = [7, 2, 5, 10, 8], M = 2 \quad s = 18(\text{split}: [7, 2, 5], [10, 8])$$

Discussion: The monotonicity property still holds. If s is a valid maximum sum of any subarray, then $s + 1$ is also a valid maximum sum of any subarray. This one still makes use of `feasible(x) -> bool` function.

12.3 Advanced: Search spaces with `enough(x) -> bool`

12.3.1 Problem [KSmallMul]: K-th smallest number in multiplication table

Problem: We define a multiplication table as a $m \times n$ matrix where

$$\text{table}[i][j] = i \times j$$

Given m, n, k , find the k -th smallest number in the multiplication table. An example may look as follows:

$$m = 3, n = 3, k = 5 \quad \text{table} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{bmatrix} \quad \text{Answer: } 3(1, 2, 2, 3, 3)$$

Discussion: For Kth-Smallest problems like this, what comes to our mind first is Heap. Usually we can maintain a Min-Heap and just pop the top of the Heap for k times. However, that doesn't work out in this problem. We don't have every single number in the entire multiplication table. Instead, we only have the height and the length of the table. If we are to apply Heap method, we need to explicitly calculate these $m \times n$ values and save them to a heap. The time complexity and space complexity of this process are both $O(mn)$, which is quite inefficient.

The search space is not clear. But we can define a function `enough(x) -> bool` that tells us if there are at least k numbers in the multiplication table that are less than or equal to x . Ofcourse, if there x satisfies k numbers, then $x + 1$ would also satisfy k numbers. We exploit this monotonicity property.

Now, we intend to beat time to be better than $O(mn)$. That means `enough(x) -> bool` should be considerably cheaper to compute. We exploit the following algorithm:

Listing 12.1: Rust implementation of enough function

```
1  fn enough(num: i32, m: i32, n: i32, k: i32) -> bool {
2      let mut count = 0;
3      // We run iteration for each row
4      for row in 1..=m {
5          // Each row has (num / row) elements less than or equal to num.
6          // However, this has to be contained within n since at most there are n columns.
7          let add = (num / row).min(n);
8          if add == 0 {
9              // Exit and do no further iterations since if there are no numbers
10             // lower than num in this row, any further rows won't have any as well.
11             break;
12         }
13         count += add;
14     }
15     count >= k
16 }
```

The above algorithm runs in $O(m)$ time.

The search space of course is $[1, m \times n]$. For every x in this search space, we can calculate the number of elements in the multiplication table that are less than or equal to x in $O(m)$ time. Hence, the runtime of this solution is $O(m \cdot \log_2(m \times n))$.

Chapter 13

Dynamic Programming

Dynamic Programming in it's crudest sense is nothing more than "clever bruteforce". What that means is while the solution eventually a large brute-force computation, the *cleverness* is in finding a sub-problem that helps solve for the larger problem instances. The key idea is "instances", not a single instance.

Take the following relation for example:

$$\begin{aligned} F(x) &= \text{Max}(F(x-1) + 1, 1) \\ F(0) &= 0 \end{aligned} \tag{13.1}$$

Here, $F(2)$ will only be used for calculating $F(3)$ and nowhere else.

On the other hand, the following relation:

$$\begin{aligned} F(x) &= \text{Max}(\text{Max}(nF(x-n) + 1) \forall n \in [0, x), 1) \\ F(0) &= 0 \end{aligned} \tag{13.2}$$

uses $F(2)$ for all calculations where $x > 2$ such as $F(3)$, $F(4)$, etc.

The cleverness in the relation 13.2 that can potentially help it get to similar asymptotics as the 13.1 is **memoization**.

13.1 Memoization

Memoization is nothing but trade-off of space to gain in time. Naive implementation of 13.2 in code will yield results with almost constant space but exponential time. The time complexity leads to:

$$O(n) = \sum_{i=0}^{n-1} O(i) + 1 \tag{13.3}$$

If however, after every solution of an instance in range $[0, k)$, the solution is kept in some constant-time accessible storage M , the problem instance k dissolves into:

$$\begin{aligned}
 O(k) &= \sum_{i=0}^{k-1} O(i) + 1 \\
 &= \sum_{i=0}^{k-1} O(1) + 1 \\
 &= k + 1
 \end{aligned}
 \tag{13.4}$$

We are going to use this idea quite generously.

13.2 Most important subproblems

Most of the times while we find ourselves looking at different problem statements, many-a-times at their core, they stem from one of the following few recursive relations:

- **O/1 Knapsack:**
- **Unbounded Knapsack:**
- **Longest Common Subsequence (LCS):**
- **Longest Increasing Subsequence (LIS):**

Alphabetical Index

asynchronous model, [16](#)

Byzantine fault, [15](#)

Consensus, [15](#)

Consistency, [14](#)

Crash fault, [15](#)

Eventual Consistency, [14](#)

Eventual Liveness, [14](#)

Fail Stop, [15](#)

FLP Impossibility Theorem,
[16](#)

Lamport Clocks, [16](#)

Liveness, [14](#)

Message Buffer, [17](#)

Node honesty, [14](#)

Omission fault, [15](#)

Partial Order, [16](#)

Strong consistency, [14](#)

Strong liveness, [14](#)

True crash fault, [15](#)